



Scrap & Sprout 설계 문서 (Design Document) (v1.0)

1. 전반적인 설계 (Overall Architecture)

구분	내용	기술 스택
Client	유저 인터페이스, 2D 렌더링, 로컬 로직 제어	Unity, C#
Game Server	재화, 인벤토리, 권한 검증, DB 트랜잭션 관리	Spring Boot, Java
AI Server	실시간 NPC 대사 생성 및 동적 시세 알고리즘	FastAPI, Python
Network	플레이어 간 위치 및 오브젝트 상태 실시간 동기화	Photon Fusion

2. 게임 엔진 설계 (Game Engine Design)

[렌더링 및 리소스]

구분	내용
URP (Universal Render Pipeline)	2D Light와 Post-Processing을 활용해 날씨(산성비, 황사)에 따른 시각적 변화를 극대화
Object Pooling	쓰레기 아이템이나 이펙트가 자주 생성/삭제되므로, 메모리 부하를 줄이기 위해 미리 객체를 생성해두고 재사용하는 풀링 시스템을 설계

[매니저 시스템]

구분	내용
GameManager	전체 게임 상태(준비, 진행, 종료) 관리.
NetworkManager	Photon 세션 연결 및 룸 매칭 제어.
UIManager	HUD, 인벤토리, AI 대화창의 노출 및 데이터 바인딩.

3. 게임 시스템 설계 (Game System Design)

[상호작용 시스템]

구분	내용
Trigger 기반 이벤트	NPC나 쓰레기에 접근 시 인터랙션 UI를 활성화하고, 'E' 키 입력 시 해당 기능을 수행합니다.
Inventory System	딕셔너리(Dictionary) 구조를 사용하여 아이템 ID와 수량을 관리하며, 서버와 동기화합니다.

[환경 및 리젠 시스템]

구분	내용
오염도 연동 알고리즘	<code>PollutionLevel</code> 변수에 따라 쓰레기 리젠 빈도와 날씨 이벤트 발생 확률을 실시간으로 계산합니다.
배터리 소모 로직	날씨 가중치(<code>\$WeatherMultiplier\$</code>)를 적용하여 실시간으로 배터리를 차감하고, 0이 될 시 행동 불능 상태로 전환합니다.

4 . 기술 문서 (Technical Specification)

[통신 프로토콜]

구분	내용
HTTP/REST	로그인, 상점 거래, 인벤토리 저장 등 '결과'가 중요한 통신에 사용.
UDP/Photon	캐릭터 이동, 아이템 수거 등 '속도'와 '동기화'가 중요한 실시간 데이터에 사용.

[데이터 무결성]

구분	내용
Server-Authoritative	모든 재화 계산과 아이템 지급은 클라이언트가 아닌 Spring 서버 에서 최종 결정합니다. 클라이언트는 요청과 연출만 담당하여 치트(해킹)를 원천 차단합니다.